

1 Midterm Review (Concepts)

Local search keeps one complete state + a move operator (it does not store the path from the start). For N -queens a board (column-per-row) encoding beats a raw grid — it removes row/column violations before search starts, and a permutation encoding removes them entirely (no repeated row or column), shrinking the infeasible space. Everything else (BFS/DFS/UCS/DLS/IDS, beam, admissibility, $\alpha\beta$) is in Foundations, the T/F Bank, and Heuristic Search below.

2 Heuristic Search (Q1 fair game)

Greedy vs A*

Greedy: $f(n) = h(n)$. A*: $f(n) = g(n) + h(n)$ = real cost so far + estimated cost remaining. A* keeps a running $f(n)$ on the frontier and always pops the smallest, so it needs the true cheapest route only, not the whole graph. g is distance already travelled; h estimates the rest.

Admissible vs Consistent

Admissible: $h(n) \leq h^*(n)$ for all n .

Consistent (monotone): $h(n) \leq c(n, n') + h(n')$ for every edge $n \rightarrow n'$; consistent $\Rightarrow$ admissible. When $h(\text{goal}) = 0$, checking direct-to-goal edges is the same as checking admissibility; the interior edges test consistency.

Dominance

If $h_2(n) > h_1(n)$ for all n and both are admissible, h_2 dominates h_1 : it is more informed and A* with h_2 expands $\leq$ the nodes of h_1 . No dominance if neither is pointwise $\geq$ the other.

Alpha-Beta Pruning

α = best (max) value MAX can guarantee so far; β = best (min) value MIN can guarantee.

Prune a subtree when $\alpha \geq \beta$ (it can no longer change the backed-up value). Best-first move ordering gives worst case $O(b^d) \rightarrow O(d^d/2)$, doubling reachable depth.

Alpha-Beta Example (left-right)

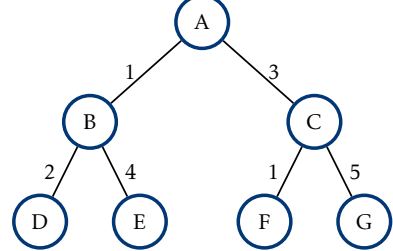
Root MAX, two MIN children L,R.

L sees 3,12,8 $\Rightarrow L=3$, so $A=3$ at root.

R sees its first child $2 \Rightarrow \beta=2$; since $A=3 > \beta=2$, prune R's remaining children (they cannot raise R above 2).

Root = $\max(3,2) = 3$.

Traversal Example (goal G)



Path costs: $g(B)=1, g(C)=3, g(D)=3, g(F)=4, g(E)=5, g(G)=8$. Ties broken alphabetically. BFS (level order): A B C D E F G. DFS (deepest, left first): A B D E C F G. UCS (increasing g): A B C D F E G; returns A–C–G, cost 8. IDS (re-run with depth limit 0,1,2): [A], [ABC], [ABDECFG] — DFS memory, BFS-style shallowest goal. PMX Procedure (permutation)

1) Pick two cut points; copy the middle segment of P_1 straight into the child. 2) For each value in P_2 's segment not yet in the child, follow the $P_1 \leftrightarrow P_2$ mapping (from the aligned segment) until you land on a free position, and place it there. 3) Fill all remaining slots directly from P_2 . Example, cuts after locus 2 and 6: $P_1=47[5318]692$, $P_2=52[4836]971$. Copy 5318; map 5 $\leftrightarrow$ 4, 3 $\leftrightarrow$ 8, 1 $\leftrightarrow$ 3, 8 $\leftrightarrow$ 6; resolve $\Rightarrow$ child = 425318976. Swap mutation keeps it a valid permutation. (Cycle crossover is not on our final.)

3 Pre-Midterm Foundations

Problem Formulation

Well-structured problem = explicit tuple $\langle S, s_0, A, T, G, c \rangle$ (states, start, actions, transition, goal test, cost); solution

= path $s_0 \rightarrow \dots \rightarrow s_g$ with $G(s_g)=1$. Ill-structured = shifting goals, hidden/incomplete constraints; needs framing + representation first. Hadamard well-posed: a solution (i) exists, (ii) is unique, (iii) is stable (depends continuously on data). Goal formulation (what matters / what to ignore) precedes problem formulation; many "algorithm failures" are really formulation failures (bad encoding, misaligned cost).

Optimization as Search

$\min_{x \in X} f(x)$ s.t. $g_i(x) \leq 0$, $h_j(x) = 0$; feasible set

$F = \{x : \text{constraints hold}\}$. Search = evaluate f , pick next via neighbourhood $N(s)$, stop on budget/target. Exact = optimal but costly; heuristic = cheap, near-optimal, no guarantee.

Constructive vs Local

Constructive: build from an empty partial state by a rule (greedy add of components). Local: start from a complete state, apply a move operator, keep if the evaluation improves. Neighbourhood $N(s)$ = states one move from s ; it defines the landscape and what "improvement" means.

Constraint Handling

Reject illegal moves; Repair/project back to feasible; or Penalty $f'(x) = f(x) + \lambda v(x)$ (v =violation amount). Keep the policy consistent — mixing them destabilises the search.

Complexity Classes

P: exact poly-time (sort, shortest path, MST). NP: a solution is verifiable in poly-time (SAT, subset-sum). NP-hard: at least as hard as any NP problem, no known poly algorithm (TSP, scheduling, placement). NP-complete: in NP and NP-hard. Blow-up: binary choices $|S|=2^n$; TSP tours $(n-1)!/2$; tree search $O(b^d)$. Verify-easy $\neq$ discover-easy.

Agent / Environment

Dimensions: observable vs partially observable; deterministic vs stochastic; static vs dynamic; discrete vs continuous; single- vs multi-agent; episodic vs sequential. Closed-world assumption = the state description is complete and trustworthy. Goal-based agent stops at the first $G(s)=1$; utility-based agent maximises $U(s) \in \mathbb{R}$.

Performance + State-Space Sizes

Completeness = finds a solution if one exists; optimality = returns the least-cost one. b =branching factor, d =depth of shallowest goal, m =max path length. Sizes: 8-puzzle $9!/2=181440$; grid $m \times n$ minus k blocked = $mn-k$; symmetric TSP $(n-1)!/2$; missionaries/cannibals state (M_L, C_L, B) with per-bank $M=0$ or $M \geq C$.

Uniformed Search Complexity

	CompL	Opt.	Time	Space
BFS	Y^b	Y^c	b^{d+1}	b^{d+1}
UCS	Y	Y	b^{1+C^*}/ϵ	b^{1+C^*}/ϵ
DFS	N	N	b^m	bm
DLS	$Y \geq d$	N	b^l	bl
IDS	Y^b	Y^c	b^d	bd
Bidir	Y^b	Y^c	$b^{d/2}$	$bd/2$

b : b finite, c : equal/non-decreasing step costs. DLS = DFS with cutoff l (complete iff $d \leq l$). Bidirectional searches from both ends and meets in the middle; needs invertible actions and a known goal state.

Minimax & Beam

Minimax: MAX maximises, MIN minimises; back up terminal utilities; optimal against an optimal opponent. Real games cut off at a depth and use an evaluation $e(p)$ (e.g. $\#open(X) - \#open(O)$); shallow cutoffs cause the horizon effect. DFS-based: Time $O(b^m)$, Space $O(m)$. Beam search keeps only the best β nodes per level $\Rightarrow$ not complete, not optimal (can discard the goal path).

4 True/False Bank (answers + why)

T = true, F = false. F A* always expands fewer nodes than UCS — a misleading heuristic can make A* expand more (e.g. h big on the good branch). T ES suits problems needing exploration; GA suits problems needing exploitation (seed GA with good guesses). F $h(s) = 1$ for all non-goal s is always admissible — near the goal the true cost can be < 1 , so it can overestimate. F BFS and IDS always return the same solution. T MDPs with small discount γ emphasise near-term rewards. T BFS is always complete even with unequal action costs (complete = finds a solution, not the cheapest). F If admissible $h_1(n)=6, h_2(n)=8$ then h_1 dominates h_2 — reversed: h_2 is closer to h^* , so h_2 dominates. T Value Iteration converges to the optimal policy (Bellman backup is a contraction).

T BFS is optimal when all edge costs are equal. F DFS always uses less memory than BFS. T An overestimating (inadmissible) heuristic can still be useful in practice (faster, if not too far off). F Hill climbing guarantees the global optimum. T SA uses randomness to escape local minima. F Tabu search uses randomness to escape minima — it is deterministic, using memory (tabu list). T Tabu tenure controls how long a move stays forbidden. F GAs operate on a single candidate at a time — they keep a population. T Selection pressure that is too high reduces population diversity (premature convergence). T No algorithm beats random search without exploiting structure (No Free Lunch). T If $h = h^*$ for all n , A* expands only optimal-path nodes (ignoring ties). T Adding the same positive constant to every arc cost can change the UCS path. F SA with constant positive T equals hill climbing (it still accepts worse moves with > 0 prob). F Alpha-beta is "more accurate" than minimax — same value, only faster. T Beam search (finite width) with an admissible h may still fail to find any solution.

5 Simulated Annealing

Accept a neighbour s' with cost change $\Delta E = E(s') - E(s)$ (minimisation):

$$P = \begin{cases} 1 & \Delta E \leq 0 \\ e^{-\Delta E/T} & \Delta E > 0 \end{cases}$$

Draw $r \sim U(0,1)$; accept the worse move iff $r \leq P$. Useful values: $e^{-1} \approx 0.37$, $e^{-2} \approx 0.14$. High T : most uphill moves accepted $\Rightarrow$ exploration (early). Low T : rarely accept worse $\Rightarrow$ exploitation, behaves like hill climbing (late). Cooling e.g. geometric $T \leftarrow \alpha T$. Init T high enough to reach almost any state (accept $\approx 60\%$ of worse moves), but not so hot it is pure random walk; use the max cost change as a guide. Stop when T is low and no better/worse moves are accepted (frozen). Constant positive T is not hill climbing (still accepts worse with > 0 prob).

SA Example (min $f(x) = x^2$, start $x=2$)

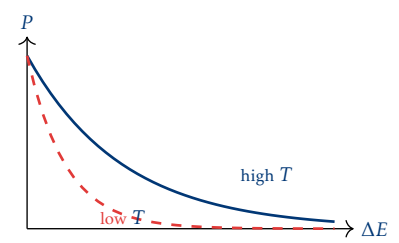
Iter 1: $x'=3$; $\Delta E = 9 - 4 = 5 > 0$, $T_1=5$:

$$P = e^{-5/5} = e^{-1} \approx 0.37; r_1 = 0.3 < 0.37 \Rightarrow \text{accept.}$$

Iter 2: $x'=4$, $\Delta E = 16 - 4 = 12 > 0$, $T_2=6$:

$$P = e^{-12/6} = e^{-2} \approx 0.14; r_2 = 0.8 > 0.14 \Rightarrow \text{reject.}$$

SA Acceptance Curve



Higher T flattens $P = e^{-\Delta E/T}$ so more uphill moves pass.

6 Tabu Search

Deterministic; escapes local optima via memory, not randomness. Each step pick the best admissible neighbour (even if worse than current), then update the tabu list and best-so-far. Short-term (recency) memory: forbid recently used move attributes for a tabu tenure, preventing immediate cycling. Long-term (frequency) memory: penalise often-used moves $\Rightarrow$ diversification into unvisited regions. Aspiration: override a tabu move if it beats the best-so-far $E(s') < E(s^*)$. Tenure too short $\Rightarrow$ cycles; too long $\Rightarrow$ over-restricted.

TS Example (min $f(x) = x^2$, tabu size 2)

Start $x=4$; sequence $4 \rightarrow 3 \rightarrow 2 \rightarrow 3$. After $4 \rightarrow 3$: tabu list [4]. After $3 \rightarrow 2$: tabu list [4,3]. At $x=2$ best candidate is $x=3$: $f(3)=9 > f(2)=4$ (not improving) and 3 is tabu; no aspiration met $\Rightarrow$ reject the move. Short-term memory blocks the $2 \rightarrow 3$ cycle; long-term memory would push the search elsewhere.

7 Ant Colony Optimization

Stigmergy: ants deposit pheromone on edges (indirect, shared memory); others are biased toward stronger trails. Solutions are built incrementally; good ones self-reinforce, poor ones fade by evaporation. Transition probability from i to feasible j :

$$P(i \rightarrow j) = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_k \tau_{ik}^\alpha \eta_{ik}^\beta}$$

τ = pheromone (learned collective experience), $\eta = 1/d$ = heuristic desirability (problem-specific), $\alpha \gg \beta$: pheromone dominates (follow reinforced edges); $\beta \gg \alpha$: heuristic dominates (greedy local). Update each iteration: evaporate then reinforce, $\tau_{ij} \leftarrow (1-\rho)\tau_{ij} + \sum_k \Delta\tau_{kij}$, $\Delta\tau_{kij} = Q/L_k$ if ant k used edge (i,j) .

Evaporation (rate ρ) is negative feedback: it removes early accidental trails and keeps the colony adaptive. Concentrating pheromone on one edge $\Rightarrow$ benefit: fast convergence/exploitation of a good edge; drawback: lost exploration / premature convergence. Applications (fair game): TSP, routing, scheduling, cell/facility assignment. A real implementation is not examined, but "which of X/Y/Z suits this problem" is.

ACO Example (ant at S, $\alpha=\beta=1$)

$\tau_{SA}=2, \tau_{SB}=1, \tau_{SC}=1; \eta_{SA}=1, \eta_{SB}=2, \eta_{SC}=1$. Weights $w_A = 2 \cdot 1 = 2$, $w_B = 1 \cdot 2 = 2$, $w_C = 1 \cdot 1 = 1$; sum = 5.

$$P(A) = \frac{2}{5}, P(B) = \frac{2}{5}, P(C) = \frac{1}{5}. \text{ A and B tie (A has pheromone, B has heuristic; equal weights).}$$

8 Particle Swarm Optimization

Each particle keeps position x_i , velocity v_i , personal best $pbest$, swarm best $gbest$. $v_{t+1} = w v_t + c_1 r_1 (pbest - x_t) + c_2 r_2 (gbest - x_t)$, then $x_{t+1} = x_t + v_{t+1}$. Inertia w : keeps previous motion / momentum $\Rightarrow$ exploration. Cognitive $c_1 r_1 (pbest - x_t)$: pull to own best $\Rightarrow$ individual exploitation. Social $c_2 r_2 (gbest - x_t)$: pull to swarm best $\Rightarrow$ collective exploitation/convergence. $r_1, r_2 \sim U(0,1)$. Clamp speed to V_{max} : too large overshoots optima; too small stalls.

Constriction PSO:

$$v_{t+1} = \chi \{v_t + c_1 r_1 (pbest - x_t) + c_2 r_2 (gbest - x_t)\}, \chi = \frac{2}{|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|}, \varphi = c_1 + c_2 > 4. \chi \text{ shrinks the whole update so velocity cannot explode } \Rightarrow \text{damped, controlled trajectories and stable convergence.}$$

PSO Example (min $f(x) = x^2$)

$x_t=4, v_t=-1, pbest=3, gbest=1, w=c_1=c_2=r_1=r_2=1$. $v_{t+1} = (-1) + (3-4) + (1-4) = -1-1-3 = -5$. $x_{t+1} = 4 + (-5) = -1$. It jumps from 4 past the minimiser 0 to $-1 \Rightarrow$ overshoot (combined pull too strong for one step).

9 Genetic Algorithms

Population $P(t) = \{x_1, \dots, x_N\}$; parallelism + diversity escape local optima. Cycle: init $\rightarrow$ evaluate $\rightarrow$ {select $\rightarrow$ crossover $\rightarrow$ mutation $\rightarrow$ replace $\rightarrow$ evaluate}. GA = variation (crossover, mutation) + selection. Genotype = encoded form operators act on; phenotype = decoded solution; fitness $f(x)$ = scalar quality, the only feedback. FPS / roulette: $p_i = f_i / \sum f_j$; higher fitness $\Rightarrow$ more likely. Crossover recombinates existing material only (no new alleles). Mutation $p_m \approx 1/L$ injects diversity. Selection exploits; crossover explores building blocks; mutation refines/diversifies. Elitism: copy top k unchanged. Increasing k raises selection pressure: + faster convergence / preserves strong building blocks; - loss of diversity, premature convergence. No Free Lunch: averaged over all problems no search beats random; gains come from representation, operators, selection, and fitness matched to the problem.

GA Example (max $f(x^2, 3\text{-bit})$)

$P = \{001, 011, 100, 110\} \rightarrow x = \{1, 3, 4, 6\} \rightarrow f = \{1, 9, 16, 36\}$. FPS ranking $110 > 100 > 011 > 001$. With crossover only (no mutation) the population converges toward 110: selection amplifies the fittest, crossover cannot restore lost diversity.

10 Differential Evolution

Mutation is population-driven and directional (not independent random noise): scaled difference of members.

Mutant vector $v = x_{r1} + F(x_{r2} - x_{r3})$, F = scale factor; the step follows the geometry of the current population. Trial vector = crossover of mutant v with target x_i (some genes from each). Selection is greedy: keep trial iff its fitness beats the target, else target survives. Contrast: GA mutation = random bit flip / small perturbation; ES mutation = Gaussian noise scaled by a step size; DE mutation = scaled difference of individuals.

11 Evolution Strategies

Individual $v = (x, \sigma)$: real object vector x plus strategy parameters σ . Mutation $x' = x + N(0, \sigma)$ (small changes more common than large). Adaptation at three levels: (1) object variables x move the search point; (2) step sizes σ set mutation scale (large $\Rightarrow$ explore, small $\Rightarrow$ fine local search); (3) orientation via covariance/correlated mutation aligns the mutation distribution with the landscape (concept only). Self-adaptation: σ is encoded in the individual and evolves with it. ($\mu + \lambda$): μ parents make λ offspring; best μ of the combined $\mu + \lambda$ survive (elitist). (μ, λ), $\lambda > \mu$: only offspring compete, parents fully replaced each generation. Classic (1 + λ)-ES: one parent, one child. Rechenberg 1/5 rule: aim for a 1/5 ratio of successful mutations. If $\varphi > 1/5$ increase σ (σ_{ci} , $c_i > 1$); if $\varphi < 1/5$ decrease (σ_{ci} , $c_i < 1$); many successes $\Rightarrow$ take bigger steps, few $\Rightarrow$ smaller steps.

Evolution Symbols (intuition)

μ = number of parents; λ = number of offspring made per generation. + means parents and children compete together (elitist, best never lost); the comma means only children compete (parents die each generation, easier to escape a bad region). σ = mutation step size / spread (large σ = big Gaussian jumps = explore; small σ = fine tuning = exploit). φ = fraction of mutations that improved the individual (drives the 1/5 rule). F = DE difference-vector scale (bigger F = larger population-driven jumps). $N(0, \sigma)$ = zero-mean Gaussian noise of width σ . p_c, p_m = crossover and per-gen mutation probabilities; L = chromosome length, so $p_m \approx 1/L$ flips about one gene per individual.

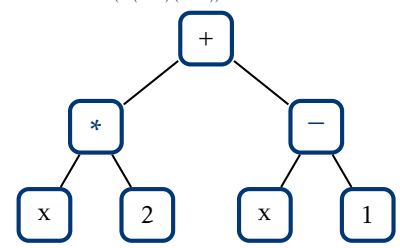
12 Memetic Algorithms

Extra step vs plain EAs = local search / local improvement applied to a generated candidate (after crossover/mutation, before/around selection). Advantage: refines the candidate in its neighbourhood $\Rightarrow$ stronger exploitation and faster convergence than stochastic operators alone. Local-search budget k /generation on a multimodal function: prefer (C) the k most diverse individuals so refinement is spread across basins and avoids premature convergence; failure mode: diverse points may sit in poor regions, wasting budget. (A) best k is also defensible (fast improvement in good basins) but risks diversity collapse.

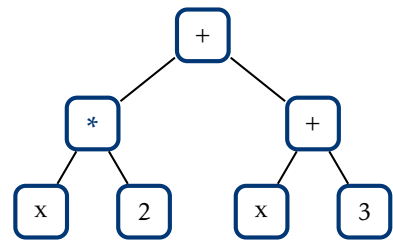
13 Genetic Programming

Evolves executable programs, not fixed vectors. Terminal set T = variables + constants (e.g. $T = \{x, R_e\}$, R_e random const in a range); function set F = operators (e.g. $F = \{+, *, \cos, \log_{10}\}$). Fitness = error vs target, e.g. sum of squared error over sample points. Closure: any function in F can take any other F/T output as an argument (untyped), so every tree is valid. Operators: subtree crossover swaps subtrees between parents; subtree mutation replaces a subtree with a new random one. Bloat = program size grows without fitness gain. Cause: accumulation of inactive code (introns) / unrestricted crossover inserting large subtrees. Control: parsimony pressure (penalise size in fitness) or a max depth/size limit.

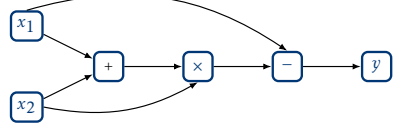
Tree-based GP: $(+ (*x2) (-x1))$



Subtree mutation: replace $(-x1)$ with $(+x3) \Rightarrow (+ (*x2) (+x3))$.



Cartesian / Graph GP
Nodes form a DAG and intermediate results are reused (not duplicated as in a tree), giving compact models.



+ gives $x_1 + x_2$; $\times$ reuses it with $x_2 \Rightarrow (x_1 + x_2)x_2$; – uses $x_1 \Rightarrow y = (x_1 + x_2)x_2 - x_1$.

GA vs GP search space
GA: fixed-length encodings (bit strings, real vectors) $\Rightarrow$ regular, uniform, easy to parameterise. GP: variable-size executable trees/graphs $\Rightarrow$ larger, irregular, rugged. Implication: GP search is harder because a small structural change can drastically change behaviour.

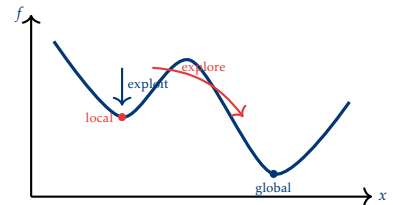
Syntax vs Semantics (GSGP)
Traditional GP evolves syntax: crossover/mutation edit program structure (tree/graph). Geometric Semantic GP evolves semantics: operators act on program behaviour/output over the data. Implication: semantic operators give a smoother, more predictable landscape (progress toward the target output), but representations can grow very large.

14 Metaheuristic Comparison

Method	Explore knob	Exploit knob
SA	high T	low T
Tabu	freq. memory	best-move pick
ACO	evaporation ρ, β	pheromone α
PSO	inertia w	cog.+soc. pull
GA	mutation, p_c	selection, elitism
DE	scale F	greedy select
ES	large σ	small σ
Memetic	EA operators	local search

Population methods (GA, ACO, PSO, DE, ES): many candidates, built-in diversity, escape local optima. Trajectory methods (HC, SA, Tabu): one state, cheap, need memory/randomness to escape.

Search Landscape (intuition)



Minimisation: exploitation descends within a basin (local opt); exploration hops over the hill to the global opt. High T /mutation/inertia widen the hops; greedy/local moves deepen the current basin.

Which Algorithm When. Continuous, smooth, differentiable: local search / gradient beats metaheuristics (No Free Lunch: no method wins everywhere). Combinatorial routing / assignment / TSP / cell placement: ACO (incremental construction over a graph) or GA with permutation encoding. Continuous multimodal function optimisation: PSO or DE (both move by population geometry).

Symbolic model / equation discovery from data: GP (evolves executable structure). Hard structure + smooth parameters: memetic (GP or GA for structure, local search for parameters). Real-vector optimisation with self-tuned steps: ES with 1/5 rule or covariance adaptation.

GP Loop (pseudocode). init random pop of trees while not done:

```
fitness[i] = error(tree[i], data)
parents = select(pop, fitness)
child = subtree_crossover(p1,p2)
child = subtree_mutation(child)
pop = replace(pop, child)
```

return best tree
Building blocks = useful subtrees; closure keeps every generated tree evaluable.

Extra Formulas & Reminders

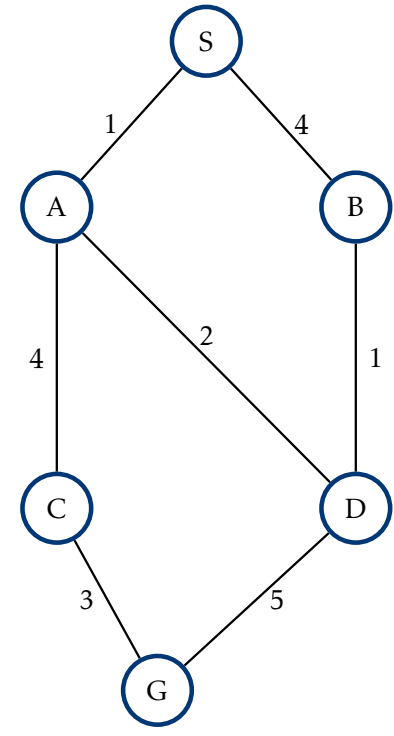
SA cooling: geometric $T_{k+1} = \alpha T_k$ ($0.8 \leq \alpha < 1$), linear $T_k = T_0 - k\beta$, logarithmic $T_k = T_0 / \log(1 + k)$ (only log guarantees asymptotic global convergence, but is impractically slow).

GA selection: roulette $p_i = f_i / \sum_j f_j$ (scale-sensitive, weak late); tournament = best of k random (pressure rises with k , needs no scaling); rank-based uses order only (scale/translation invariant); SUS = evenly spaced pointers; elitism keeps best E .

All metaheuristics are anytime: termination (max generations/evaluations, convergence, or hitting a target) never implies optimality. UCS and A* goal-test on expansion, not generation. Alpha-beta returns exactly the minimax value. Greedy, beam, and hill climbing are neither complete nor optimal.

Winter 2026 Final (with answers). This closely resembles the upcoming final; each part states what is asked, then the answer.

Q1 Search [10] – graph. Start S , goal G , ties broken alphabetically. Edges: $S \rightarrow A=1$, $S \rightarrow B=4$, $A \rightarrow C=4$, $A \rightarrow D=2$, $B \rightarrow D=1$, $C \rightarrow G=3$, $D \rightarrow G=5$.



Heuristics: $h_1(S, A, B, C, D, G) = (4, 3, 4, 2, 3, 0)$; $h_2 = (5, 4, 5, 3, 5, 0)$.
(a) Asks: run UCS; give expansion order, path, cost. By g : $S(0); A(1), B(4)$; expand A : $C(5), D(3)$;

$D(3); G(8); B(4); C(5); G(8); G$. Order $SADBCG$; path $S \rightarrow A \rightarrow D \rightarrow G$; cost 8.
(b) Asks: run A^* with h_2 ; expansion, path, cost. $f=g+h_2$: $S(5); A(5); B(9); C(8); D(8); G(8)$. Order $SACD G$; path $S \rightarrow A \rightarrow D \rightarrow G$; cost 8.
(c) Asks: is h_1 admissible and consistent? Justify. True costs $h^* = (8, 7, 6, 3, 5, 0) \Rightarrow$ all $h_1 \leq h^*$ (admissible); every edge $h_1(n) \leq c + h_1(n')$ (e.g. $S: 4 \leq 1+3$) $\Rightarrow$ consistent.
(d) Asks: is h_2 admissible and consistent? Yes to both (all edges hold, some tight).
(e) Asks: does one dominate; implication? $h_2 \geq h_1$ everywhere, both admissible $\Rightarrow h_2$ dominates $\Rightarrow A^*$ with h_2 expands $\leq$ nodes.

Q2 SA & Tabu [10]. (a) Asks: SA min x^2 from $x=2$; moves $x'=3(T=5, r=0.3)$, $x'=4(T=6, r=0.8) \Rightarrow$ accept?; and how/when SA explores vs exploits. Move! $\Delta=5$, $S=P \cdot e^{-1} = 0.37 > 0.3 \Rightarrow$ accept; move! $\Delta=12$, $P=e^{-2} = 0.14 < 0.8 \Rightarrow$ reject. High T early accepts worse (explore); T late $\Rightarrow$ greedy (exploit).
(b) Asks: Tabu size 2 from $x=4$, moves $4 \rightarrow 3 \rightarrow 2 \rightarrow 3$; list after each accepted move; accept 3 at $x=2$; short vs long memory. Lists $[4]$ then $[4, 3]$. At $x=2$, 3 gives $9 > 4$ and is tabu, no aspiration $\Rightarrow$ reject. Short-term list stops cycling; long-term frequency memory diversifies.

Q3 ACO & PSO [20]. (a) Asks: ant at S with $\tau_{SA, SB}, SC=2, 1, 1$, $\eta=1$, $2, 1$, $\alpha=\beta=1$; compute P to $A/B/C$; most likely; effect of $\alpha \gg \beta$ and $\beta \gg \alpha$; one benefit+drawback of pheromone concentrating on one edge. Weights $2, 2, 1 \Rightarrow P = \frac{2}{5}, \frac{2}{5}, \frac{1}{5}$; A, B tie. $\alpha \gg \beta \Rightarrow A$; $\beta \gg \alpha \Rightarrow B$. Concentrating: +fast exploitation, –premature convergence.
(b) Asks: PSO min x^2 , $x=4$, $v=-1$, $pbest=3$, $gbest=1$, coeffs 1; compute v_{t+1}, x_{t+1} ; convergence/overshoot/oscillatory?; role of the 3 terms; role of χ . $v=-1$ $1-3=-5$, $x=-1 \Rightarrow$ overshoot (passes 0). Inertia=momentum/explore; cognitive=pull to $pbest$; social=pull to $gbest$. χ scales the whole update $\Rightarrow$ stable, non-diverging.

Q4 GA & GP [20]. (a) Asks: GA max x^2 , $P=\{001, 011, 100, 110\}$; rank by FPS; will crossover-only converge (no mutation)?; how elitism k affects pressure (+/-). $f=\{1, 9, 16, 36\} \Rightarrow$ rank $110>100>011>001$. Yes converges: selection amplifies fit, crossover cannot restore diversity. Larger k =more pressure: +faster convergence, –diversity loss.
(b) Asks: draw tree of $(+x2)(-x1)$; subtree-mutate $(-x1) \rightarrow (+x3)$, draw; define bloat + one cause + one control; identify the shown program-representation type + write its program; compare GA vs GP search space; explain syntax vs semantics (GSGP) + one implication. Trees drawn in the GP section. Bloat = size grows without fitness gain (cause: introns/unrestricted crossover; control: parsimony pressure or depth cap). Figure = Cartesian/graph GP with reuse

$\Rightarrow y = (x_1 + x_2)x_2 - x_1$. GA = fixed-length regular space; GP = variable-size structural $\Rightarrow$ GP harder. Syntax=structure, semantics=behaviour; GSGP smooths the landscape but grows programs.

Q5 DE, Memetic, ES [20]. (a) Asks: how DE mutation differs from other EAs; explain mutant/trial vectors + selection. DE uses scaled population differences $v = x_{r1} + F(x_{r2} - x_{r3})$ (directional) vs GA flip / ES Gaussian. Offspring: mutant $v \rightarrow$ crossover with target = trial $\rightarrow$ greedy select (keep trial if better).
(b) Asks: extra step memetic adds, when applied, one advantage; for multimodal + budget k choose best- k /random- k /diverse- k + failure mode. Adds local search on candidates (after variation) $\Rightarrow$ exploitation. Pick (C) diverse- k (spreads refinement, avoids premature convergence); failure: diverse points may be in poor regions.
(c) Asks: list ES adaptation levels + effect. Object variables (where), step sizes σ (how far: big explore/small refine), orientation/covariance (which directions).

Q6 Memetic control design [20]. Setup: learn $T_{in}(t+1) = f(x_1, x_2, x_3, u)$ from data (Phase 1), then pick u minimising $J = \sum_k [(T_{in} - T_{ref})^2 + \lambda u^2]$ (Phase 2).
(a) Asks: memetic strategy — what is explored globally, refined locally, when local search runs. Globally evolve model structure with GP; locally refine parameters with local search, applied after structural candidates are evaluated.
(b) Asks: two algorithms to discover structure of f +

their search space. Tree GP (expression trees of F, T); Cartesian GP (DAGs with reuse).
(c) Asks: how one evolves structure toward better fit. Score each tree on data (MSE), select better fits, apply subtree crossover/mutation over generations.
(d) Asks: local search to pick control sequence $u(l, t+H)$ and how it uses f . HC/SA/Tabu: simulate T_{in} with f , plug into f , tweak one entry, keep if f drops.
(e) Asks: why Phase 1 or Phase 2 alone is insufficient. Model alone gives no policy; control alone tunes on an inaccurate model $\Rightarrow$ need both, so a memetic split fits.

Fall 2024 Final (with answers). Q1 true/false items live in the T/F Bank above; the worked parts follow.

Q1 traversal + heuristics + $\alpha\beta$ + PMX. (b) Asks: for a cost-labelled tree (goal 1) give BFS, DFS, UCS, IDS visit order. Method: BFS level order; DFS deepest-first; UCS by cumulative cost; IDS = depth-limited DFS with a rising limit until I appears (see Traversal Example).
(c) Asks: 3-state problem, heuristics I–IV — circle admissible? consistent?; and any dominance. Only II inadmissible ($h(B)=4 > h^*(B)=3$); consistency on edge $A-B$ (cost 2): $I: 4-1=3 > 2$ no, $III: 4-3=1 \leq 2$ yes, $IV: 5-2=3 > 2$ no. III, IV no dominance; IV dominates I .
(d) Asks: run minimax with $\alpha\beta$ on a game tree, ties left-to-right; mark pruned branches and each node's value/bound. Method: back up max/min, prune a subtree once $\alpha \geq \beta$; label never-visited leaves as pruned.
(e) Asks: first offspring by (i) Cycle Crossover, (ii) PMX (cuts after locus 2 and 6) on $p_1=475318692, p_2=524836971$. PMX $\Rightarrow 425318976$ (procedure above). CX excluded (not on our final).
Q2 explore/exploit controls + GA math. (a) Asks: for SA, GA, PSO, GP, Tabu, name the mechanism that controls explore vs exploit. GA: temperature T . GA: mutation/crossover rate + selection pressure. PSO: inertia w vs cognitive/social pull. GP: crossover/mutation on tree structure + selection. Tabu: tenure (short) vs frequency memory (long).
(b) Asks: genotype vs phenotype (+example); GA vs ES; ES self-adaptation. Genotype = encoded (bit string); phenotype = decoded solution (real χ). ES uses real vectors + Gaussian self-adaptive mutation and $(\mu, \lambda)(\mu, \lambda)$; self-adaptation evolves σ inside the individual.

(c) Asks: binary GA, pop N , mut rate ρ , n bits — (i) P (no bit mutates in a generation), (ii) min ρ for that $P \leq P_{none}$, (iii) numeric for $N=n=100$, $P_{none}=0.0001$. (i) $(1-\rho)^{Nn}$; (ii) $\rho = 1 - P_{none}^{1/(Nn)}$; (iii) $\rho \approx 0.0092$.
Q3 airline crew GA. Setup: 3 planes, 5 crews, one crew/plane/day, no crew > 2 days in a row.
(a) Asks: chromosome? 3 genes over $\{A..E\}$ (position = plane), or a 5-gene binary work/off string. (b) Alphabet + size? 5 letters, or 2 (binary). (c) Fitness? reward valid coverage, penalise rule violations. (d) How many solutions; is GA needed; scaling? grows combinatorially; GA pays off as planes/crews grow.

Q4 SA + ES. (a) Asks: guidelines for initial/final SA temperature. Init high enough to reach any state (accept $\approx 60\%$ worse), not pure-random; final need not be 0, stop when frozen. (b) Asks: ways to cut SA compute cost. $e^{-\Delta/T}$ lookup table, non-exponential acceptance, incremental cost updates. (c) Asks: describe ES + its mutation operator. (x, σ) , mutate $x' = x + N(0, \sigma)$, small changes more common. (d) Asks: (μ, λ) vs (μ, λ) : +: parents+children compete (elitist); comma: only children survive, parents replaced. (e) Asks: Rechenberg 1/5 rule + why + params. Keep success ratio $1/5 > 1/5$ grow σ , $< 1/5$ shrink; Schwefel $c_d=0.82, c_1=1.22$.

Q5 GP for regression. Asks: for fitting a piecewise $f(x)$, give (a) terminal set, (b) function set, (c) fitness, (d) a GP tree + a mutation on it, (e) the closure property + example. $T = \{x, \Re\}$ ($\Re$ const $\approx [-5, 5]$); $F = \{+, -, \times, \cos, \log, | \cdot |$; fitness = sum of squared error to target; mutation swaps a random subtree; closure = any $f \in F$ accepts any $g \in F$ (untyped), so trees stay valid.

Q6 ACO for min $f(x) = x^2 - 2x - 11$ on $(0, 3)$. Asks: use ACO (4 ants, evaporation 0.2) for 2 iterations. Discrete $(0, 3)$ into candidate points; ants pick with $P \propto \tau^\alpha \eta^\beta$, η larger where f is smaller (min at $x=1, f=-12$); each iteration evaporate $\tau \leftarrow 0.8\tau$ then deposit $\propto 1/f$; after 2 iterations pheromone concentrates near $x=1$.

Q7 PSO explained. Asks: (a) explain the velocity equation + all parameters; (b) identify inertia/cognitive/social; (c) show (pseudocode/steps) how PSO solves a given problem. $v_{t+1} = wv_t + c_1 r_1 (pbest-x) + c_2 r_2 (gbest-x)$,

$x_{t+1} = x_t + v_{t+1}$; w inertia (explore), c_1, c_2 cognitive/social (exploit), $r_1, r_2 \sim U(0, 1)$. Loop: init particles, evaluate f , update $pbest/gbest$, update v then x , until done; return $gbest$.

Spring 2021 Final (with answers). Q1 true/false items are in the T/F Bank above; worked parts follow.

Q1 search + crossover. (b) Asks: which algorithm best manages worst-case time/space, and how? Iterative Deepening — DFS-like linear memory (no stored frontier) yet never builds paths longer than the shortest solution.

(c) Asks: reorder words of an unknown language using a “ridiculousness” score — search-space-size?; define a neighbourhood + 2 examples; will HC / Tabu always find a valid sentence? Space = $n!$; local optimum = swap two words’ positions. HC no (local optima); Tabu no either (only optimal as $t \rightarrow \infty$).
(d) Asks: first offspring by Cycle Crossover and by PMX (cuts after locus 2, 6). PMX $\Rightarrow 425318976$; CX excluded (not on our final).

(e) Asks: two parents differ in every bit; 1-point crossover at random $c \in [1, n]$ — P (children are clones)? = $1/n$ (only when the cut is at an extreme end).
(f) Asks: when is exploration vs exploitation more desirable? Exploit when good guesses exist (seed a GA); explore when none exist (use ES).

Q2 MDP treasure ($y=1$). Setup: collect treasures and return to start within a time budget (else utility $-\infty$); a boulder blocks a cell, present w.p. 0.5, observed only when adjacent. Asks: $V^*(S_9), V^*(S_{13}), V^*(S_{\infty})$ and the min state-space size. $V^*(S_9) = 20$;

$V^*(S_{13}) = \frac{40+30}{2} = 35$ (peek, else fall back);
 $V^*(S_{\infty}) = 10+20+80 + \frac{40}{2} + \frac{30}{4} + \frac{160}{4} = 185$; size = $MN \cdot 3^B \cdot 2^X$ (position $\times$ boulder present/absent/unknown $\times$ treasure taken/not).

Q3 graph search + h_3 . Setup: undirected graph, start A goal G , h_1 consistent, h_2 not. (a) Asks: which listed paths each of DFS/BFS/UCS/A* (h_1) / A* (h_2) could return. DFS any; BFS the shallowest $A \rightarrow B \rightarrow D \rightarrow G$, $A \rightarrow C \rightarrow D \rightarrow G$; UCS & both A* the optimal $A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$. (b) Asks: $h_3(B)$ values making h_3 admissible / consistent. Admissible $0 \leq h_3(B) \leq 12$; consistent $9 \leq h_3(B) \leq 10$. (c) Asks: $h_3(B)$ making A* expand A, C, B, D in order. $12 < h_3(B) < 13$.

Q4 SA + sensor placement. (a) Asks: SA initial/final temperature guidelines. High init (accept $\approx 60\%$ worse, not random); final need not be 0, stop when frozen. (b) Asks: SA speed-ups. Lookup table, non-exponential acceptance, incremental cost. (c) Asks: place 5 cameras on 8×8 (gate cells cost $\times 2$) — representation, neighbourhood, objective, then run SA ($T_0=500$, linear -50). Rep = $m \times 3$ array $(x, y, orient)$; neighbours = move/swap/rotate a camera; objective (min) = $2 \times (\text{uncovered gate}) + (\text{uncovered other})$; accept if better else if $e^{-\Delta/T} > r$.

Q5 6-Queens. Setup: one queen per column, Eval = number of non-attacking pairs. (a) states? 6^6 . (b) successors per state? 30 (move one queen in its column). (c) Eval of the shown board? 9. (d) FPS probs from Eval $\{9, 12, 11, 8\}$? $\frac{9}{40}, \frac{12}{40}, \frac{11}{40}, \frac{8}{40}$. (e) which GA operator is like the successor function? Mutation.

Q6 EA/ES/ACO. (a) Asks: EA mutation vs PSO velocity update difference. Mutation = random perturbation; velocity is directed by $pbest/gbest$. (b) Asks: how ES adapts. Self-adaptation: σ encoded in the individual, evolves with it. (c) ACO TSP (ant at C , visited A , $\alpha=\beta=1$, $\eta=1/d$): asks P to $A/B/D$, update counts, density vs quantity, $\alpha=0/\beta=0$, variant names. $P(A)=0$;

$P(B) = \frac{0.98/6}{0.98/6 + 0.67/3} = 0.422$; $P(D)=0.578$. Updates/iter (5 ants): step-by-step 15, delayed 5, offline 1. Density adds Q (ignores length); quantity adds Q/d_{ij} .
 $\alpha=0 \Rightarrow$ greedy heuristic; $\beta=0 \Rightarrow$ pure pheromone. Variants: adaptive per-ant=Near-Param-Free ACS; bounded range=MMAS; GA-tuned=ACSGA; pseudo-random/elitist=ACSS; delayed=ant=AS.
Q7 PSO topologies + permutation. (a) Asks: match 5 descriptions to topologies. nearest-in-space(costly)=physical neighbors; fastest spread, easily stuck=star; matrix-adjacent=social neighbors; grid-of-4=Von Neumann; slowest spread, avoids minima=ring. (b) Asks: permutation PSO update ($w=\frac{1}{3}, c=2, r=0.5$) — V_2, X_2 . Velocity is swaps: $V_2 = ((1, 3), (2, 6), (4, 7))$; $X_2 = X_1 + V_2 = [2, 6, 1, 7, 5, 4, 8, 3]$.